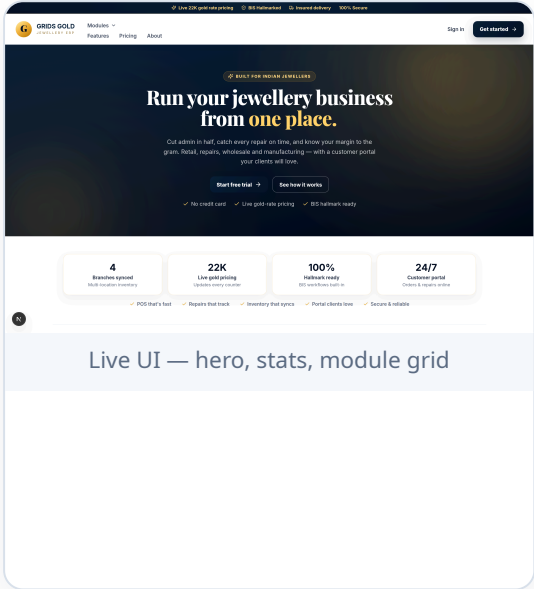


GET /

Marketing Homepage

Primary file: app/page.tsx · Components: LandingNav, ScrollReveal, TypewriterText

Purpose: Public marketing site. Static content arrays drive sections (hero, stats, modules, pricing). All CTAs route to /login.



```
app/page.tsx — Hero + routing

// Client component — uses animations & interactive
nav
"use client";

export default function LandingPage() {
  return (
    <section className="landing-hero">
      <h1>Run your jewellery business from{" "}
        <TypewriterText
          phrases={["one place.", "every branch."]}
        />
      </h1>
      <Link href="/login?signup=1">
        Start free trial
      </Link>
    </section>
  );
}
```

CODE TO EXPLAIN

```
app/lib/landingModules.ts — data-driven module registry

export const moduleGroups = [
  {
    label: "Daily operations",
    items: [
      { title: "POS & Sales", href: "/pos" },
      { title: "Repairs", href: "/repairs" },
    ],
  },
];
// moduleHighlights = flatMap(groups) → used in nav + homepage grid
```

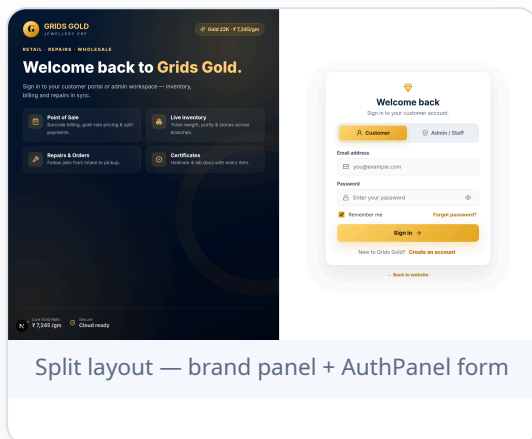
Say while demoing: This is a *Client Component* because it uses hooks and browser APIs (Intersection Observer, typewriter timers). Content is config arrays mapped to JSX — no API fetch, so Vercel can prerender it statically. `next/link` gives client-side navigation to auth without a full reload.

GET `/login?signup=1`

Login & Sign Up

Files: `app/login/page.tsx` · `app/components/AuthPanel.tsx` · `app/lib/store.tsx`

Purpose: Single auth entry for *Customer* and *Admin*. URL query `?signup=1` toggles mode. Submit writes to React Context store, then `router.push()` redirects.



`app/login/page.tsx` — URL-driven mode

```
function LoginContent() {
  const params = useSearchParams();
  const isSignup = params.get("signup") === "1";

  return (
    <AuthPanel
      initialMode={isSignup ? "signup" : "signin"}
    />
  );
}

// Wrapped in <Suspense> — required for
useSearchParams()
```

CODE TO EXPLAIN — AUTH SUBMIT FLOW

`app/components/AuthPanel.tsx` — validation + redirect

```
async function handleSubmit(e) {
  e.preventDefault();
  if (isAdmin) {
    // Demo credential gate
    signup({ role: "admin" });
    router.push("/dashboard");
    return;
  }
  if (isSignup) signup({ name, email, role });
  else login(email, role);
  router.push(isAdmin ? "/dashboard" : "/portal");
}
```

Say while demoing: Two axes of state — mode (signin/signup) and role (customer/admin). Admin uses hardcoded demo credentials today; Supabase path exists when `NEXT_PUBLIC_SUPABASE_*` env vars are set. `signup()` / `login()` update `currentUser` in Context and persist to `localStorage` key `gg_state_v4`. Sign out clears user and returns to `/login`.

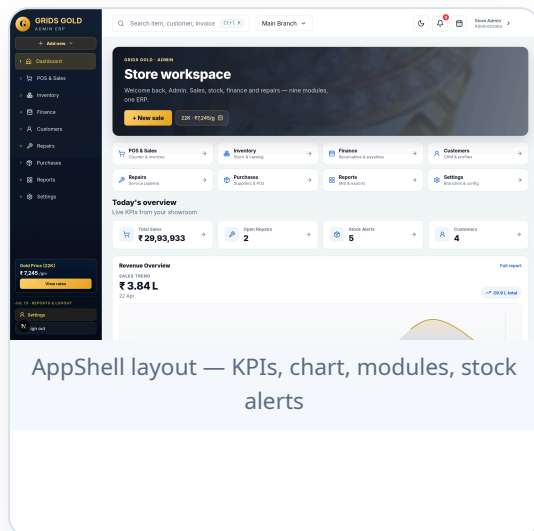
- **Admin demo:** `nancy2005nov@gmail.com` / `nancy` → `/dashboard`
- **Customer:** any email + `signup` → `/portal`
- **StoreProvider** in `layout.tsx` wraps all routes

`GET /dashboard`

Admin Dashboard

Files: `app/dashboard/page.tsx` • `app/components/AppShell.tsx` • `app/lib/analytics.ts`

Purpose: Admin home after login. Reads live ERP data from `useStore()`, derives KPIs/charts with `useMemo`, renders inside collapsible `AppShell` sidebar layout.



AppShell layout — KPIs, chart, modules, stock alerts

`app/dashboard/page.tsx` — data from store

```
export default function DashboardPage() {
  const { invoices, items, rates, repairs } =
    useStore();

  const trend = useMemo(
    () => salesTrendFromInvoices(invoices),
    [invoices]
  );

  return (
    <AppShell>
      <SmoothSalesChart points={trend} />
      <TopItemsTable items={...} />
    </AppShell>
  );
}
```

CODE TO EXPLAIN — INTERACTIVE SVG CHART

`app/dashboard/page.tsx` — smooth line + mouse tracking

```
function smoothLinePath(coords) {
  // Cubic Bézier between invoice trend points
  path += ` C ${cp1x} ${cp1y}, ${cp2x} ${cp2y}, ${p2.x} ${p2.y}`;
}

<div
  onMouseMove={(e) => pickIndex(e.clientX)}
  className="dash-chart"
>
  <svg viewBox="0 0 100 100">
    <path d={linePath} />
  </svg>
</div>
```

Say while demoing: Dashboard is a composed Client Component. `useStore()` supplies invoices/items/rates; `analytics.ts` pure functions derive chart series. `useMemo` avoids recomputing on unrelated re-renders. Chart uses SVG paths (no chart library) with mouse/touch handlers mapping X position to nearest data point. `AppShell` provides sidebar nav from `adminNav.ts` (9 modules).

- **Price formula:** $\text{weight} \times \text{goldRate}[\text{karat}] + \text{making} + \text{stoneValue}$
- **Persisted state:** `localStorage` until Supabase backend is wired
- **Build:** `pnpm build prerenders` all routes for Vercel